LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 3 – KONSEP DASAR PEMROGRAMAN BERORIENTASI OBJEK



NAMA : Elok Elysia Dewi

NIM : 103132400013

KELAS : S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. **Pemrograman Berorientasi Objek (OOP)** adalah cara membangun sistem dari komponen-komponen yang mengemas data dan fungsi menjadi satu kelompok, yang dapat mewarisi sifat dari komponen lain dan saling berinteraksi.
2. **Karakteristik utama OOP** terdiri dari abstraksi (menemukan hal esensial pada objek berupa atribut dan perilaku), enkapsulasi (memaketkan data dan method serta menyembunyikan rincian implementasinya), pewarisan (membuat kelas baru dengan mewarisi kelas yang sudah ada), polymorphism ("satu interface banyak aksi"), dan message (komunikasi antar objek lewat pemanggilan method).
3. **Kelas (class)** adalah cetak biru atau rancangan dari objek yang berisi atribut dan method, dan dari satu kelas dapat dibuat banyak objek.
4. **Objek (object)** adalah hasil ciptaan (instansiasi) dari suatu kelas yang dibuat dengan keyword new, misalnya Manusia objMns1 = new Manusia();.
5. **Field** adalah atribut penyimpan data dalam kelas, yang terbagi menjadi class field (memakai static, dimiliki kelas, dan dipakai bersama seperti Circle.PI) dan instance field (tanpa static, dimiliki masing-masing objek, seperti jari-jari r pada tiap lingkaran).
6. **Method** adalah fungsi atau prosedur yang memodularisasi program dengan memisahkan tugas dalam kelas, disebut fungsi bila mengembalikan nilai (return) dan prosedur bila tidak mengembalikan nilai (void).
7. **Cara melewatkan argumen ke method** ada dua, yaitu pass by value untuk tipe primitif (yang disalin adalah nilainya sehingga variabel asal tidak berubah) dan pass by reference untuk array dan objek (yang dikirim adalah alamat memori sehingga perubahan di dalam method memengaruhi variabel asal).
8. **Keyword this** adalah referensi ke objek yang sedang berlaku, dipakai untuk membedakan field dengan parameter yang namanya sama dan untuk memanggil constructor lain dalam kelas yang sama.
9. **Hal penting lainnya** adalah constructor bermodifier private hanya dapat dikenali di dalam kelasnya sendiri sehingga objek hanya bisa dibuat dari dalam kelas itu, dan method main harus bersifat public static.

B. Guided

1. Kucing

```
package abstraksi;


public class Kucing {
```

```
    String nama;
    String ras;

    void bersuara() {
        System.out.println(nama + ras + "meow");
    }
}
```

2. Abstraksi

```
package abstraksi;

public class Abstraksi {
    public static void main(String[] args) {
        Kucing aren = new Kucing();

        aren.nama = "Aren Gentong";
        aren.ras = "Mujaer";
        aren.bersuara();
    }
}
```

Output :

```
run:
Aren GentongMujaermeow
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. Rekening

```
package encapsulation;

public class Rekening {
```

```
    private int saldo = 0;

    public void tambahSaldo(int jumlah) {
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo() {
        System.out.println("Saldo Anda: " + saldo);
    }
}
```

4. Encapsulation

```
package encapsulation;

public class Encapsulation {
    public static void main(String[] args) {
        Rekening rek = new Rekening();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}
```

Output :

```
run:
Saldo berhasil ditambahkan
Saldo Anda: 20000000
BUILD SUCCESSFUL (total time: 0 seconds)
```

5. Buku

```
public class Buku {

    private String pengarang;
    private String judul;

    private Buku() {
        this("Rumah Kita", "GoodBles");
// this ini digunakan untuk memanggil konstruktor yang
// menerima dua parameter
    }
```

```
    private Buku(String judul, String pengarang) {
        this.judul = judul;
        this.pengarang = pengarang;
    }

    private void cetakKeLayar() {
        System.out.println("Judul : " + judul + " Pengarang : "
                + pengarang);
    }

    public static void main(String[] args) {
        Buku a, b;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}
```

Output :

```
run:
Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles
BUILD SUCCESSFUL (total time: 4 seconds)
```

6. Circle

```
public class Circle {

    public static final double PI = 3.14159;

    public static double radiansToDegrees(double rads) {
        return rads * 180 / PI;

    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }
```

```
}
```

7. DemoLagu

```
class Lagu {

    private String pencipta;
    private String judul;

    public void IsiParam(String judul, String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }

    public void cetakKeLayar() {
        if (judul == null && pencipta == null) {
            return;
        }
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}

public class DemoLagu {

    public static void main(String[] args) {
        Lagu a = new Lagu();
        a.IsiParam("God Will Make A Way", "Don Moen ");
        a.cetakKeLayar();
    }
}
```

Output :

```
run:
Judul : God Will Make A Way, pencipta : Don Moen
BUILD SUCCESSFUL (total time: 1 second)
```

8. TestPass

```
public class TestPass {

    // 1. Atribut
    int i, j;

    // 2. Konstruktor
    TestPass(int a, int b) {
```

```
        i = a;
        j = b;
    }

    // 3. Method Pass by Value (primitif)
    void calculate(int m, int n) {
        m = m * 10;
        n = n / 2;
    }

    // 4. Method Pass by Reference (objek)
    void calculate(TestPass e) {
        e.i = e.i * 10;
        e.j = e.j / 2;
    }

    // 5. Entry point untuk menjalankan program
    public static void main(String[] args) {
        int x, y;
        TestPass z;
        z = new TestPass(50, 100);
        x = 10;
        y = 20;

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        // passed by value
        z.calculate(x, y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        // passed by reference
        z.calculate(z);
        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
    }
}
```

Output :

```
run:
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50
BUILD SUCCESSFUL (total time: 2 seconds)
```

C. Unguided

1. Soal Unguided 1

Soal — Pembersihan dan Analisis Data Suhu Mingguan

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Ekspektasi Output:

```
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
```

Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
```

---

Jawaban :

```
package unguided;

public class TugasModul2 {

    public static void main(String[] args) {
        // Deklarasi konstanta KKM
        final double KKM = 75.0;
```
```

```
        // Array 1 Dimensi untuk menyimpan nama mahasiswa
        String[] namaMahasiswa = {"Andi", "Budi", "Citra"};

        // Array 2 Dimensi Rectangular (3 baris, 2 kolom) untuk nilai Modul 1 dan Modul 2
        double[][] nilaiModul = {
            {80.0, 85.0}, // Nilai Andi
            {70.0, 65.0}, // Nilai Budi
            {90.0, 90.0} // Nilai Citra
        };

        // Header output
        System.out.println("REKAP NILAI PRAKTIKUM");
        System.out.println("KKM: " + KKM);
        System.out.println();

        // Perulangan untuk mengakses data array, menghitung rata-rata, dan evaluasi status
        for (int i = 0; i < namaMahasiswa.length; i++) {
            double rataRata = (nilaiModul[i][0] + nilaiModul[i][1]) / 2.0;
            String status;

            // Percabangan untuk menentukan kelulusan
            if (rataRata >= KKM) {
                status = "LULUS";
            } else {
```

```
            status = "REMEDIAL";
        }

        // Menampilkan data mahasiswa dan hasil evaluasi
        System.out.println("Mahasiswa " + (i + 1) + ": " + namaMahasiswa[i]);
        System.out.println("Nilai Modul 1 : " + nilaiModul[i][0]);
        System.out.println("Nilai Modul 2 : " + nilaiModul[i][1]);
        System.out.println("Rata-rata     : " + rataRata);
        System.out.println("Status        : " + status);
        System.out.println();
    }
  }
}
```

Output :

```
run:
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
BUILD SUCCESSFUL (total time: 1 second)
```

Penjelasan :
Program ini membuat class PengolahSuhu yang menyimpan data suhu 7 hari secara terlindungi (private), lalu mencari hari yang datanya kosong (-1.0), mengisinya dengan rata-rata suhu hari sebelum dan sesudahnya, dan menghitung rata-rata suhu, sambil menunjukkan bahwa array di main ikut berubah karena array bertipe reference.

Import

```
import java.util.Arrays;
import java.util.Locale;
```

Arrays dipakai untuk mencetak isi array dengan Arrays.toString(). Locale dipakai agar angka desimal tampil dengan titik (29.39), bukan koma.

Deklarasi class dan field

```
public class PengolahSuhu {
    private static final double NILAI_KOSONG = -1.0;
    private double[] suhuHarian;
```

- NILAI_KOSONG adalah class field: static (dipakai bersama semua object) dan final (nilainya tidak bisa diubah). Fungsinya sebagai penanda data kosong.
- suhuHarian adalah instance field: tiap object punya array sendiri. Keduanya private agar tidak bisa diakses langsung dari luar class.

Constructor

```
public PengolahSuhu(double[] suhuHarian) {
    this.suhuHarian = suhuHarian;
}
```

Dijalankan saat object dibuat dengan new. this.suhuHarian adalah field milik object, sedangkan suhuHarian di kanan adalah parameter. Array yang disimpan adalah array yang sama dengan di main (referensi, bukan salinan).

tampilkanData()

```
public void tampilkanData() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            System.out.println("Hari " + (i + 1) + " : (kosong)");
        } else {
            System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + "\u00B0C");
        }
    }
}
```

Mengulang semua elemen array. Jika nilainya sama dengan NILAI_KOSONG, dicetak (kosong), selain itu dicetak suhunya. i + 1 dipakai karena index mulai dari 0 sedangkan hari mulai dari 1. \u00B0 adalah kode Unicode untuk simbol derajat °.

cariIndexKosong()

```
public int cariIndexKosong() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            return i;
        }
    }
    return -1;
}
```

Mencari elemen yang bernilai NILAI_KOSONG. Begitu ketemu, langsung mengembalikan index-nya (return i). Jika tidak ada, mengembalikan -1 sebagai tanda "tidak ditemukan".

isiDataKosong()

```
public void isiDataKosong() {
    int index = cariIndexKosong();
    if (index != -1) {
        suhuHarian[index] = (suhuHarian[index - 1] + suhuHarian[index + 1]) / 2;
    }
}
```

Memanggil cariIndexKosong() untuk mendapat index yang kosong. Jika ada (index != -1), nilainya diganti dengan rata-rata hari sebelumnya (index - 1) dan hari sesudahnya (index + 1). Pada data soal: (26.8 + 31.4) / 2 = 29.1.

hitungRataRata()

```
public double hitungRataRata() {
    double total = 0;
    for (int i = 0; i < suhuHarian.length; i++) {
        total += suhuHarian[i];
    }
    return total / suhuHarian.length;
}
```

Menjumlahkan seluruh suhu ke total, lalu membaginya dengan banyaknya data (7). Harus dipanggil setelah data diisi supaya -1.0 tidak ikut terhitung.

main: persiapan data dan object

```
double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };
PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);
```

Membuat array suhu mentah (hari ke-4 kosong), lalu membuat object pengolah dan menyerahkan array itu ke constructor.

main: tampil data awal dan cari index

```
System.out.println("=== Data Suhu Awal ===");
pengolah.tampilkanData();
System.out.println("Index hari kosong (dimulai dari 0): " +
pengolah.cariIndexKosong());
```

Menampilkan data apa adanya, lalu menampilkan index hari kosong, yaitu 3.

main: isi data dan tampil hasil

```
pengolah.isiDataKosong();
pengolah.tampilkanData();
System.out.printf(Locale.US, "Rata-rata : %.2f\u00B0C%n",
pengolah.hitungRataRata());
```

Mengisi hari yang kosong, menampilkan data yang sudah bersih, lalu mencetak rata-rata. %.2f berarti 2 angka di belakang koma, dan %n berarti baris baru.

main: bukti array ikut berubah

```
System.out.println(Arrays.toString(suhuHarian));
```

Mencetak array dari variabel main (bukan lewat object). Hasilnya sudah berisi 29.1 padahal main tidak pernah mengubahnya langsung. Penyebabnya, array adalah tipe reference: main dan object memegang alamat memori yang sama, jadi perubahan lewat object ikut terlihat di main .

Penjelasan Output

Bagian output	Asalnya	Penjelasan
Data Suhu Awal Hari 4 : (kosong)	tampilkanData() pertama	Hari 4 bernilai -1.0 (sama dengan NILAI_KOSONG), jadi dicetak (kosong).
Index hari kosong: 3	cariIndexKosong()	-1.0 ada di index 3. Index mulai dari 0, jadi Hari 4 = index 3.
Data Suhu Setelah Pengisian Hari 4 : 29.1°C	isiDataKosong() lalu tampilkanData()	Hari 4 diisi rata-rata tetangganya: (26.8 + 31.4) / 2 = 29.1.
Rata-rata : 29.39°C	hitungRataRata()	205.7 / 7 = 29.3857, dibulatkan 2 desimal (%.2f) menjadi 29.39.
Array di main: [..., 29.1, ...]	Arrays.toString(suhuHarian)	Array di main ikut berubah karena main dan object memegang satu array yang sama.

Array adalah tipe reference, jadi constructor menyimpan alamat array itu, bukan salinannya. Perubahan lewat object otomatis terlihat dari main.

D. Kesimpulan

1. Class sebagai cetak biru. Class PengolahSuhu menggabungkan data (suhuHarian) dan perilaku (tampilkanData(), cariIndexKosong(), isiDataKosong(), hitungRataRata()) dalam satu kesatuan, sehingga pengolahan data suhu lebih terstruktur dan bisa dipakai ulang untuk object lain.
2. Enkapsulasi menjaga data. Field dibuat private, sehingga data hanya bisa diubah lewat method public dan terlindung dari perubahan sembarangan dari luar class.
3. Dua jenis field punya fungsi berbeda.
    a. Instance field (suhuHarian) dimiliki masing-masing object, sehingga datanya bisa berbeda tiap object.
    b. Class field (NILAI_KOSONG, static final) dimiliki class, dipakai bersama, dan nilainya tetap. Nilai -1.0 cukup ditulis sekali.
4. Constructor dan this mengisi field. Saat parameter dan field bernama sama, this.suhuHarian = suhuHarian dipakai untuk membedakan keduanya.
5. Array adalah tipe reference. Constructor menyimpan alamat array yang sama dengan milik main, bukan salinannya. Karena itu perubahan lewat object (isiDataKosong()) ikut terlihat di main, sesuai konsep pass by reference di modul.
6. Hasil program benar. Data kosong di Hari 4 berhasil diisi dengan 29.1°C (rata-rata 26.8 dan 31.4), dan rata-rata suhu data bersih adalah 29.39°C.

Dengan OOP, data suhu beserta cara mengolahnya dikemas dalam satu class yang aman (enkapsulasi), menggunakan class field untuk aturan bersama dan instance field untuk data tiap object, dan perilaku array sebagai reference menjelaskan kenapa perubahan di dalam object ikut terlihat di main.